



Rapport de Projet

Self Hosted Cloud Implementation

Hugo CLAVEILLE

Encadré par Gilles MENEZ

Master 1 Informatique

12 mai 2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	State of the Art	1
1.2.1	De l'abstraction des ressources au cloud computing	1
1.2.2	Systèmes distribués et accès distant	2
1.2.3	Informatique orientée services	2
1.2.4	Virtualisation et conteneurisation	2
1.2.5	Orchestration et automatisation	2
1.2.6	Modèles du cloud computing	3
1.2.7	DevOps et Infrastructure as Code	3
1.3	Problem statement	3
2	Matériel et Méthodes	5
2.1	Infrastructure disponible	5
2.2	Solutions candidates	5
2.2.1	Méthodologie de sélection	5
2.2.2	Solutions étudiées	6
2.3	Solution retenue : Dokploy	6
2.4	Application déployée : MNIST cloud classification	7
2.4.1	Déploiement avec Dokploy	8
2.5	Monitoring et gestion des logs	9
2.5.1	Tests de performance	9
2.6	Scalabilité avec Dokploy	9
2.6.1	Scaling horizontal	9
2.6.2	Scaling horizontal via les répliquas	10
2.6.3	Autoscaling via l'API de Dokploy	11
2.7	Orchestration dans Dokploy	11
2.8	Amélioration du RNN server	12
2.8.1	Amélioration de l'autoscaling	12
3	Résultats et Discussion	13
3.1	Résultats	13
3.1.1	Déploiement et monitoring	13
3.1.2	Performance under load	13
3.1.3	Scalabilité avec Dokploy	13
3.1.4	Autoscaling	14
3.2	Discussion	14
3.2.1	Adéquation aux besoins	14
3.2.2	Limites matérielles	14
3.2.3	Limites de Dokploy	14
3.3	Perspectives	15
4	Conclusion	16

Résumé

Ce rapport présente la conception, le déploiement et l'évaluation d'une solution de cloud self-hosted dans le cadre du projet DS4H.

L'objectif principal est de choisir et fournir une alternative locale au service "Render" utilisé jusqu'alors dans le cours d'IoT. Ceci permettra aux étudiants de déployer et de gérer des applications IoT dans un modèle PaaS (Platform As A Service).

La spécification "Self Hosted" doit permettre à la formation de ne plus dépendre d'un service cloud externe. En ce sens, on rejoint ainsi une démarche de cloud privé.

Cette étude vise aussi à explorer quelques cas d'utilisation permettant de tester plus précisément certaines possibilités du cloud retenu. Le rapport détaille donc les choix technologiques, les étapes de mise en œuvre, ainsi que les résultats obtenus en termes de performance et de scalabilité.

1. Introduction

1.1 Contexte

Dans le cadre du projet **DS4H**, il m'a été confié la mission de concevoir et déployer une **solution self-hosted** pour remplacer le service **Render** utilisé dans le cours d'**IoT**.

L'objectif est de fournir une alternative locale, souveraine et maîtrisée, permettant aux étudiants et enseignants de déployer, gérer et monitorer des applications IoT sans dépendre d'un service cloud externe.

Ce besoin, bien que formulé dans un contexte pédagogique, s'inscrit en réalité dans une problématique plus large : celle du **cloud computing** et de ses différentes formes d'exploitation.

En effet, remplacer un service comme Render ne consiste pas uniquement à déployer une application sur un serveur.

Cela implique de reproduire, à plus petite échelle, un ensemble de mécanismes caractéristiques du cloud moderne :

- abstraction de l'infrastructure,
- déploiement automatisé,
- isolation des applications,
- gestion multi-utilisateurs
- élasticité des ressources.

Autrement dit, ce projet revient à concevoir une **plateforme de type PaaS**¹ simplifiée, adaptée à un contexte contraint (une seule machine, ressources limitées, utilisateurs non experts).

Afin de comprendre les enjeux techniques et les choix réalisés dans ce travail, il est donc nécessaire de replacer cette problématique dans son contexte historique et conceptuel.

1.2 State of the Art

1.2.1 De l'abstraction des ressources au cloud computing

L'idée d'abstraire les ressources informatiques précède largement l'émergence du cloud computing moderne. Dès les premiers systèmes d'exploitation, comme Multics, apparaît le concept de « computer utility », dans lequel la puissance de calcul est perçue comme un service accessible à la demande, indépendamment de la machine physique sous-jacente [11].

Ce changement de paradigme introduit un principe fondamental du cloud : l'utilisateur interagit avec une interface abstraite, tandis que l'infrastructure réelle est masquée et gérée séparément.

Dans le cadre de ce projet, cette abstraction est essentielle, car les utilisateurs (étudiants) doivent pouvoir déployer leurs applications sans avoir à manipuler directement l'infrastructure système.

1. Platform as a Service

1.2.2 Systèmes distribués et accès distant

L'accès distant aux ressources de calcul est une idée fondatrice de l'informatique. Dès 1940, *George Stibitz* démontre la possibilité de piloter une machine à distance via une liaison télégraphique [10].

Cette logique a évolué vers des paradigmes distribués, comme le Grid Computing, dans lesquels plusieurs machines collaborent pour exécuter des tâches complexes [13].

Ces approches introduisent des concepts clés tels que le partage de ressources, la transparence réseau et l'exécution distribuée, qui sont au cœur des plateformes cloud modernes.

Aujourd'hui, une application n'est plus nécessairement liée à une seule machine, mais peut être répartie sur plusieurs instances, améliorant ainsi la scalabilité et la tolérance aux pannes.

1.2.3 Informatique orientée services

L'émergence du Software as a Service (SaaS) marque une rupture majeure dans la manière de consommer les logiciels. À partir de la fin des années 1990, des entreprises comme Salesforce proposent des applications accessibles via un navigateur web [19].

Ce modèle introduit l'idée que les différentes couches de l'informatique (logiciel, plateforme, infrastructure) peuvent être fournies sous forme de services [16].

Cette approche est directement liée à l'objectif de ce projet, qui consiste à remplacer Render, une plateforme de type PaaS², par une solution auto-hébergée offrant des fonctionnalités similaires de déploiement et de gestion d'applications.

1.2.4 Virtualisation et conteneurisation

La virtualisation est une technologie clé ayant permis l'essor du cloud computing. Elle permet d'exécuter plusieurs environnements isolés sur une même machine physique, optimisant ainsi l'utilisation des ressources [20].

Cependant, les machines virtuelles présentent certaines limites, notamment en termes de consommation de ressources et de temps de démarrage. Ces contraintes ont conduit à l'émergence des conteneurs, qui offrent une isolation plus légère en partageant le noyau du système hôte [17].

Les conteneurs sont aujourd'hui devenus l'unité standard de déploiement dans les environnements cloud, car ils permettent d'assurer la portabilité et la reproductibilité des applications.

Dans ce projet, cette approche est centrale, puisque la solution retenue (Dokploy) repose sur l'utilisation de conteneurs Docker pour le déploiement et la gestion des applications.

1.2.5 Orchestration et automatisation

L'orchestration des conteneurs,

2. Platform as a Service

1.2.6 Modèles du cloud computing

Les infrastructures cloud peuvent être classées selon différents modèles de déploiement :

- **Cloud public** : les ressources sont fournies par un prestataire externe et mutualisées entre plusieurs utilisateurs.
- **Cloud privé** : l'infrastructure est dédiée à une seule organisation.
- **Cloud hybride** : combinaison des deux approches.

Par ailleurs, les modèles de service définissent le niveau d'abstraction proposé [16, 14] :

- **IaaS** (Infrastructure as a Service) : mise à disposition de ressources matérielles virtualisées.
- **PaaS** (Platform as a Service) : environnement de déploiement d'applications.
- **SaaS** (Software as a Service) : accès à des logiciels complets.

La solution développée dans ce projet peut être interprétée comme une forme simplifiée de PaaS, déployée sur une infrastructure self-hosted, s'apparentant à un cloud privé.

1.2.7 DevOps et Infrastructure as Code

Le cloud computing moderne repose fortement sur l'automatisation. L'Infrastructure as Code (IaC) permet de définir et de gérer l'infrastructure à l'aide de fichiers de configuration, garantissant la reproductibilité et la cohérence des environnements [18].

Parallèlement, les pratiques DevOps visent à réduire la séparation entre développement et exploitation, notamment grâce à l'automatisation des déploiements et à l'intégration continue [9].

Ces concepts se retrouvent dans les outils modernes de déploiement, qui permettent de connecter un dépôt de code à une infrastructure capable de construire et déployer automatiquement une application.

Dans ce projet, ces principes sont directement appliqués à travers l'utilisation de Dokploy, qui automatise le déploiement d'applications à partir de dépôts Git et fournit un environnement cohérent pour les utilisateurs.

1.3 Problem statement

Le besoin identifié dans le cadre du projet ne se limite pas à un simple remplacement fonctionnel du service Render.

Il s'agit en réalité de déployer une plateforme capable de reproduire, dans un contexte contraint, certaines propriétés fondamentales d'un environnement cloud.

Le problème peut être formulé de la manière suivante :

Comment déployer une plateforme de type PaaS, auto-hébergée, permettant à des utilisateurs non experts de déployer et gérer leurs applications, tout en respectant des contraintes fortes de ressources, de simplicité et de maintenabilité ?

Cette problématique introduit plusieurs contraintes majeures.

Tout d’abord, l’infrastructure disponible est limitée : le système doit fonctionner sur un nombre restreint de machines, avec des ressources finies en termes de CPU, mémoire et stockage.

Contrairement aux fournisseurs de cloud public, il n’est pas possible de s’appuyer sur une élasticité quasi infinie.

Ensuite, les utilisateurs ciblés (les étudiants) ne disposent pas nécessairement de compétences en administration système ou en DevOps.

La solution doit donc masquer la complexité technique et proposer une interface simple et accessible.

Enfin, le projet s’inscrit dans un contexte pédagogique, impliquant une facilité de déploiement, de maintenance et de reproduction de l’infrastructure, idéalement par une seule personne.

À partir de ces contraintes, plusieurs objectifs techniques peuvent être définis :

- **Simplicité de déploiement** : permettre aux utilisateurs de déployer une application avec un minimum d’interactions techniques.
- **Isolation des applications** : garantir que les différentes applications déployées ne perturbent pas l’environnement global.
- **Gestion multi-utilisateurs** : offrir un système de comptes permettant de séparer les usages.
- **Monitoring basique** : fournir un accès aux logs et à l’état des applications.
- **Maîtrise des ressources** : limiter l’empreinte système afin de préserver les capacités de l’infrastructure.

Ces objectifs serviront de critères d’évaluation pour comparer les différentes solutions techniques envisagées dans la suite de ce rapport, et justifier le choix final de l’architecture retenue.

2. Matériel et Méthodes

2.1 Infrastructure disponible

L'ensemble des expérimentations a été réalisé sur un serveur privé virtuel (VPS) tournant sous Debian GNU/Linux 12 (Bookworm), disposant des ressources suivantes :

- **CPU** : 2 vCPU Intel Xeon E5-2680 v4 @ 2.40 GHz
- **RAM** : 4.2 Go (environ 1.5 Go disponibles en conditions normales)
- **Stockage** : 25 Go

Cette contrainte est déterminante, car elle impose des limites strictes en termes de mémoire, de puissance de calcul et d'espace disque.

Contrairement aux environnements cloud publics, aucune élasticité automatique n'est possible automatiquement, dans le cadre d'un VPS, il faut faire une demande de changement de plan pour augmenter les ressources, et dans le cas d'une machine physique, il faut acheter du matériel supplémentaire pour faire du scaling vertical.

Dans ce contexte, toute solution retenue doit être :

- légère en consommation de ressources,
- simple à déployer sur une machine unique,
- capable de coexister avec plusieurs applications sans saturation rapide.

Ces contraintes ont fortement influencé les choix technologiques présentés dans la suite de cette section.

2.2 Solutions candidates

Plusieurs solutions de cloud self-hosted ont été étudiées afin de répondre aux besoins identifiés dans la section précédente.

L'objectif n'est pas de construire une infrastructure industrielle, mais de sélectionner une solution adaptée à un contexte pédagogique, avec un compromis entre fonctionnalités et complexité.

POURQUOI manquer d'ambitions ? alors que tu n'es pas encore tombé sur une seule restriction ...

2.2.1 Méthodologie de sélection

Les solutions ont été évaluées selon les critères suivants :

- **Facilité de déploiement** : installation rapide sur une machine unique.
- **Support du déploiement applicatif** : intégration avec Git et Docker.
- **Gestion multi-utilisateurs** : capacité à isoler les projets.
- **Interface utilisateur** : accessibilité pour des utilisateurs non experts.
- **Consommation de ressources** : compatibilité avec les contraintes du VPS.

2.2.2 Solutions étudiées

OpenStack [8] se définit comme un système d'exploitation cloud qui contrôle de larges pools de ressources de calcul, de stockage et de réseau à travers un datacenter, gérées via des APIs ou un tableau de bord [8]. Sa complexité et ses besoins matériels le rendent toutefois inadapté à une infrastructure limitée.

Kubernetes (k3s) [7] est une distribution Kubernetes certifiée, hautement disponible, conçue pour des charges de production dans des environnements contraints en ressources. Il se conditionne en un unique binaire de moins de 70 Mo, réduisant les dépendances nécessaires à son installation et son exécution [7]. Malgré sa légèreté, il introduit une complexité d'orchestration qui dépasse les besoins du projet.

Dokku [6] est une plateforme PaaS open-source extensible qui s'exécute sur un serveur unique. Il permet de construire des applications à la volée depuis un `git push`, via un Dockerfile ou en détectant automatiquement le langage avec des Buildpacks [6]. Cette approche reste cependant limitée en termes d'interface utilisateur et de gestion multi-utilisateurs.

CapRover [1] se présente comme un gestionnaire de déploiement d'applications et de serveur web, extrêmement simple d'utilisation, s'appuyant sur Docker, nginx, Let's Encrypt et NetData derrière une interface épurée [1]. Il reste néanmoins limité dans la gestion fine des utilisateurs.

Coolify [2] est une plateforme PaaS open-source qui simplifie l'auto-hébergement d'applications, de bases de données et de services sur son propre serveur, compatible avec un large éventail de langages et de frameworks [2]. Sa consommation importante de ressources le rend incompatible avec l'infrastructure disponible.

Solutions	Interface Web	Multi Users	Git deploy	Complexité	Ressources	Adéquation
OpenStack	✓	✓	~	Très élevée	Élevées	×
Kubernetes (k3s)	~	~	~	Élevée	Moyennes	×
Dokku	×	Limitée	✓	Faible	Faibles	~
CapRover	✓	Limitée	✓	Moyenne	Faibles	~
Coolify	✓	✓	✓	Faible	+20 Go	×
Dokploy	✓	✓	✓	Faible	Faibles	✓

TABLE 2.1 – Comparaison des solutions de cloud self-hosted évaluées

2.3 Solution retenue : Dokploy

Au regard des critères définis, **Dokploy** a été retenu comme solution principale. Ce choix repose sur plusieurs éléments :

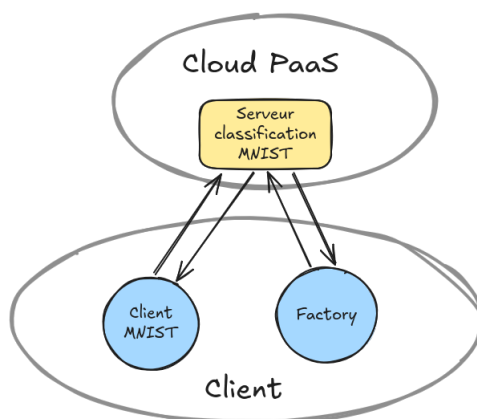
- Une installation simple en une commande sur un serveur Linux.
- Une interface web permettant une prise en main rapide.
- Une intégration native avec Git pour le déploiement continu.
- Un support du multi-utilisateur avec gestion des permissions.
- Une consommation de ressources compatible avec le VPS.

Dokploy repose sur l'utilisation de conteneurs Docker pour isoler les applications et sur un reverse proxy¹ pour exposer les services.

Il constitue ainsi une implémentation légère d'une plateforme de type PaaS, adaptée à un contexte pédagogique.

2.4 Application déployée : MNIST cloud classification

Afin d'évaluer la solution retenue, une application de test a été déployée.



Il s'agit d'un serveur de classification de texte basé sur un réseau de neurones récurrent (RNN), permettant de simuler une charge réaliste.

Le RNN a été entraîné sur le dataset MNIST, qui contient des images de chiffres manuscrits, afin de pouvoir classer des images envoyées par les clients.

L'application est composée de trois éléments :

- **RNN_factory.py** : génération et entraînement du modèle.
- **RNN_server.py** : serveur exposant une API de classification. L'API est définie de la manière suivante :
 - **POST /predict** : reçoit une image encodée en base64 et retourne le chiffre prédit ainsi que les probabilités associées.
 - **POST /upload** : permet de téléverser un nouveau modèle au format `.keras` qui est immédiatement activé.
 - **GET /models** : liste les modèles disponibles en base de données.

1. Reverse proxy : Traefik décrit plus en détail plus tard

- `POST /models/<id>/activate` : active un modèle existant par son identifiant.
- `DELETE /models/<id>` : supprime un modèle non actif.
- `GET /metrics` : retourne les métriques système en temps réel (CPU, RAM, charge).
- **RNN_client.py** : client permettant de charger des images et de soumettre des requêtes à l'API. Ce client sera également utilisé pour réaliser des tests de charge et mesurer les performances du serveur en fonction du nombre de requêtes simultanées.

Cette application a été choisie car elle permet :

- de simuler une charge CPU significative,
- de tester la stabilité du déploiement,
- d'évaluer la capacité de montée en charge de la plateforme.

2.4.1 Déploiement avec Dokploy

Dans un premier temps, un nouveau projet est créé au sein de Dokploy sous l'intitulé "RNN-Classification".

À l'intérieur de ce projet, une application est ensuite ajoutée et nommée "RNN-Server".

Plusieurs approches de déploiement sont envisageables :

- Il est possible d'utiliser une image Docker préalablement construite et publiée sur un registre (tel que Docker Hub ou GitHub Container Registry), que Dokploy peut déployer directement.
- Une autre option consiste à fournir directement les fichiers sources à Dokploy afin qu'il construise l'image à partir d'un Dockerfile.
- Enfin, il est également possible de connecter un dépôt Git (GitHub, GitLab, etc.) afin d'automatiser le processus de déploiement lors de chaque mise à jour du code.

Dans le cadre de ce projet, la troisième approche a été retenue afin de bénéficier d'un flux de travail plus intégré et automatisé.

Un dépôt Git est donc créé pour le projet, dans lequel sont ajoutés les fichiers `RNN_factory.py`, `RNN_client.py` et `RNN_server.py`.

Un fichier `Dockerfile` est ensuite défini afin de spécifier les étapes de construction de l'image de l'application, ainsi qu'un fichier `requirements.txt` listant les dépendances nécessaires, telles que TensorFlow et Flask.

Du côté de Dokploy, le projet est configuré pour se connecter au dépôt Git et pour utiliser le Dockerfile afin de construire automatiquement l'image à chaque nouvelle version poussée sur le dépôt.

La commande de lancement du serveur est également définie, à savoir : `gunicorn -bind 0.0.0.0:5000 RNN_server:app`.

Enfin il faut paramétrer le domaine sur lequel va répondre l'api, (ici `rnn.dokpoly.claveille.fr`) et le port pour Traefik (ici : 5000).

Une fois la configuration terminée, le déploiement peut être initié.

Dokploy se charge alors de construire l'image Docker, de la déployer sur le cluster, puis d'exposer le service via une URL accessible.

2.5 Monitoring et gestion des logs

Dokploy offre un dashboard centralisé permettant de visualiser l'état des applications déployées, ainsi que les logs générés par les conteneurs (voir Figure A.4).

Cela permet de suivre en temps réel les performances de l'application, d'identifier les erreurs éventuelles et de diagnostiquer les problèmes de déploiement.

2.5.1 Tests de performance

Une fois le serveur déployé, un script de stress test a été créé à partir de `RNN_client.py` pour envoyer des requêtes de classification et mesurer les performances du serveur en termes de latence, de débit et de taux d'erreur. Le script `RNN_stress_test.py` possède plusieurs paramètres configurables :

- **Nombre de requêtes** : le nombre total de requêtes à envoyer au serveur.
- **Nombre de workers** : le nombre de processus parallèles à utiliser pour envoyer les requêtes.
- **Nom du fichier de sortie** : le nom du fichier CSV dans lequel les résultats seront enregistrés.

Les résultats sont présentés dans les figures A et A, et seront discutés plus en détail dans la section 3.1.2.

2.6 Scalabilité avec Dokploy

Dokploy permet de faire deux types de scaling :

1. Le **scaling vertical**, qui consiste à augmenter les ressources allouées à une instance (CPU, RAM, etc.) afin d'améliorer ses performances.

Cette approche est limitée par les ressources physiques disponibles sur la machine hôte.

2. Et le **scaling horizontal**, qui consiste à ajouter des machines supplémentaires au cluster.

Cette approche permet de répartir la charge entre plusieurs nœuds, mais nécessite une configuration plus complexe et une gestion de l'orchestration des conteneurs pour assurer la cohérence et la disponibilité des applications déployées.

2.6.1 Scaling horizontal

Le scaling horizontal, aussi appelé server scaling², permet d'ajouter des machines supplémentaires au cluster afin de répartir la charge entre plusieurs nœuds.

Architecture du cluster. En ajoutant des machines supplémentaires, la répartition des tâches est la suivante :

2. <https://docs.dokploy.com/docs/core/cluster#server-scaling-methods>

- Une machine principale, dite *manager*, où est installé Dokploy et qui gère le cluster, et une ou plusieurs machines secondaires, dites *workers*. Le manager orchestre les tâches et répartit les conteneurs à déployer entre les workers en fonction de la charge et des ressources disponibles.
- Chaque worker est connecté au registry Docker, ce qui lui permet de récupérer les images nécessaires pour exécuter les applications déployées.
- Le manager est également responsable de la repartition du trafic entrant entre les différentes instances de l'application via le reverse proxy Traefik, assurant ainsi une forme de load balancing.

Ordonnancement des tâches. Dokploy s'appuie sur Docker Swarm [5] pour l'orchestration du cluster. Le manager utilise l'algorithme *Spread* [4] pour distribuer les tâches entre les workers : il place chaque nouveau conteneur sur le nœud disposant du plus grand nombre de ressources disponibles (CPU, mémoire), en privilégiant le nœud ayant le moins de conteneurs en cours d'exécution en cas d'égalité [4].

Contraintes de placement. Il est possible de personnaliser ce comportement via des contraintes de placement (*placement constraints*), qui sont des règles strictes forçant l'exécution d'un service sur des nœuds portant certaines étiquettes (*labels*), par exemple pour réserver certains workers à des charges de travail spécifiques [3]. Des préférences de placement (*placement preferences*) permettent en complément une répartition souple, par exemple entre zones de disponibilité [3].

2.6.2 Scaling horizontal via les réplicas

Nous allons donc tester la scalabilité de notre solution en augmentant le nombre d'instances du serveur.

Dokploy permet de faire cela facilement depuis le dashboard, en ajustant le nombre de conteneurs déployés pour l'application.

Pour cela, Dokploy nécessite un **Docker Registry**, qui est un service de stockage et de distribution d'images Docker. Il joue le rôle d'un dépôt centralisé depuis lequel chaque instance peut récupérer l'image de l'application à déployer. Sans ce registry, les différents conteneurs ne pourraient pas accéder à une version commune et cohérente de l'image.

Nous utilisons ici le **GitHub Container Registry (GHCR)**, qui s'intègre nativement avec notre dépôt GitHub et ne nécessite pas d'infrastructure supplémentaire. Une fois le registry configuré dans Dokploy, il suffit d'ajuster le nombre de réplicas depuis l'onglet **Advanced** de l'application. Traefik, le reverse proxy intégré à Dokploy, se charge alors automatiquement de répartir le trafic entre les différentes instances.

Pour les évaluations suivantes, les paramètres utilisés pour le stress test sont les suivants :

- **Nombre de requêtes** : 100
- **Nombre de workers** : 10

Le nombre de réplicas est ensuite ajusté de 1 à 3, afin d'observer l'impact sur la latence et l'utilisation des ressources. Voir les résultats dans la section 3.1.3, Figure A.3.

2.6.3 Autoscaling via l'API de Dokploy

Dokploy expose une API REST complète, accessible depuis `/api` et documentée via une interface Swagger disponible à `<domaine>:3000/swagger`. L'authentification s'effectue par clé API, générée depuis le profil administrateur.

Cette API permet notamment de modifier le nombre de réplicas d'une application via l'endpoint `application.update`, en passant le champ `replicas` dans le corps de la requête :

Sur cette base, un script d'autoscaling a été développé. Son principe est le suivant : il interroge périodiquement (ici toutes les 30 secondes) l'utilisation CPU de la machine hôte, et ajuste dynamiquement le nombre de réplicas de la manière suivante :

- Si l'utilisation CPU dépasse 80%, le script augmente le nombre de réplicas de 1, jusqu'à un maximum de 2, car au-delà la machine est saturée.
- Si l'utilisation CPU descend en dessous de 30%, le script réduit le nombre de réplicas de 1, jusqu'à un minimum de 1.
- Si l'utilisation CPU est comprise entre 30% et 80%, le nombre de réplicas reste inchangé.

Ce script constitue une forme rudimentaire d'autoscaling horizontal, analogue au principe du *Horizontal Pod Autoscaler* (HPA)[15] de Kubernetes, mais implémenté sans dépendance à un orchestrateur complexe.

Il est important de noter que Dokploy ne propose pas nativement de mécanisme d'autoscaling automatique pour les applications non-Compose³. Ce script pallie donc cette limite de manière pragmatique, en exploitant l'API existante.

La donnée ainsi collectée est agrégée par fenêtre de 2 secondes afin d'augmenter la lisibilité des courbes(A.5).

2.7 Orchestration dans Dokploy

Dokploy gère l'orchestration des conteneurs via Docker Swarm[12].

1. **En cas de crash d'une instance**, Docker Swarm détecte la défaillance et si Docker Engine ne parvient pas à redémarrer le conteneur sur le même nœud, Swarm le recrée automatiquement sur un autre nœud du cluster (dans le cas d'une configuration multi-machines) ou dans le même nœud si les ressources le permettent afin de respecter le nombre de réplicas défini pour l'application.
2. **En cas de saturation des ressources**, Docker Swarm ne dispose pas de mécanismes avancés de gestion de la surcharge.

La seule solution consiste à ajuster manuellement le nombre de réplicas ou à augmenter les ressources disponibles sur les nœuds du cluster. Il existe une solution d'autoscaling via l'API de Dokploy, mais elle n'est pas native et nécessite la mise en place d'un script externe comme décrit dans la section 2.6.3.

3. Cette fonctionnalité est explicitement listée comme manquante dans le suivi des issues du projet.

Cependant, il est important de noter que l'API de Dokploy ne fournit pas d'informations détaillées sur la charge ou la latence des applications, ce qui limite la capacité à implémenter un autoscaling réactif et efficace basé sur des métriques précises.

Qui le fait autre que dokploy ? et si l'app mesurait ça ?

2.8 Amélioration du RNN server

Afin d'essayer de tirer parti des fonctionnalités de Dokploy, notamment la gestion d'une base de données, le code du RNN server a été amélioré afin qu'il puisse sauvegarder le modèle entraîné dans une base de données, et mettre à jour le modèle à chaud pour éviter de devoir réentraîner à chaque changement.

Sauvegarde du modèle dans une base de données

Premièrement, il a fallu deployer un service PostgreSQL dans le projet Dokploy et créer une table `models` qui stocke les modèles sérialisés en binaire (BYTEA) avec les champs `id`, `name`, `data`, `created_at` et `is_active`.

Ensuite, il a fallu modifier le code du RNN server pour qu'il puisse se connecter à la base de données, sauvegarder le modèle entraîné dans la table `models` via la route `/upload` et sélectionner le modèle actif via `/models/<id>/activate`.

Problème lié aux réplicas

Comme mentionné précédemment, le RNN server est déployé sur plusieurs replicas, et traefik se charge de répartir le trafic entre les différentes instances.

Cependant, cela pose un problème : quand on formule la requête `/upload` ou `/models/<id>/activate` afin de mettre à jour le modèle actif, la requête n'est traitée que par une seule instance.

Cela signifie que les autres instances ne sont pas informées du changement de modèle, et continuent à utiliser l'ancien modèle.

Pour résoudre ce problème une solution a été mise en place : avant chaque prédiction, chaque replica consulte la BDD pour vérifier si le modèle actif a changé (`active_model_id`), et recharge uniquement si nécessaire. Tous les replicas restent ainsi synchronisés sans communication directe entre eux.

2.8.1 Amélioration de l'autoscaling

Afin d'améliorer le script d'autoscaling décrit dans la section 2.6.3, une nouvelle fonctionnalité a été ajoutée : si le nombre de réplicas atteint le maximum autorisé (2 dans notre cas) et que l'utilisation CPU dépasse toujours 80%, le script fait une requête à l'endpoint `/models/<id>/activate` pour activer un modèle plus léger (moins de neurones, moins de couches) qui consomme moins de ressources.

PAS ENCORE FAIT A FAIRE

3. Résultats et Discussion

3.1 Résultats

3.1.1 Déploiement et monitoring

Le déploiement de l'application RNN-Server via Dokploy s'est déroulé sans accroc, confirmant la bonne intégration du pipeline de construction d'image Docker à partir du dépôt Git.

L'application est accessible à l'URL configurée (`rnn.dokploy.claveille.fr`) et répond correctement aux requêtes de classification.

Le dashboard de Dokploy permet de suivre en temps réel les logs et les métriques système, offrant une visibilité précieuse sur le comportement de l'application sous charge.

3.1.2 Performance under load

La première phase d'expérimentation a consisté à évaluer les performances de l'application en conditions de charge, sur un seul replica, afin d'identifier les limites de la configuration initiale.

Comme nous pouvons l'observer dans les figures A et A, une seule instance du serveur consomme en moyenne 50% des ressources CPU disponibles sur la machine, et environ 70% de la RAM indépendamment du nombre de workers utilisés pour le stress test. Nous pouvons également observer que plus le nombre de workers augmente, plus la latence moyenne et p95¹ augmente, ce qui indique que le serveur est rapidement saturé et que les requêtes s'accumulent en file d'attente, ce qui dégrade les performances.

Cela illustre les limites de notre infrastructure : avec une seule instance la capacité de traitement est rapidement saturée.

3.1.3 Scalabilité avec Dokploy

Nous avons ensuite testé la scalabilité de notre solution en augmentant le nombre d'instances du serveur via les réplicas de Dokploy, afin d'observer l'impact sur la latence et l'utilisation des ressources.

Comme illustré dans la figure A.3, le passage de 1 à 2 replicas améliore légèrement les performances : la latence moyenne est réduite de 1458 ms à 899 ms, les deux vCPU disponibles étant alors pleinement exploités.

En revanche, l'ajout d'un troisième replica provoque une dégradation sévère : le CPU atteint 96% de charge, la RAM 93.6%, et la latence p95 bondit de 1309 ms à 8257 ms.

On peut supposer que cette saturation des ressources entraîne un recours au swap disque, ce qui expliquerait ces temps de réponse extrêmes. Sur cette machine, 2 replicas constituent donc le point d'équilibre optimal, au-delà duquel la contention des ressources dégrade fortement les latences.

1. P95 (95e percentile de latence) : valeur en dessous de laquelle se situent 95% des temps de réponse observés. Elle permet d'évaluer les performances dans les cas lents sans être trop influencée par les valeurs extrêmes.

3.1.4 Autoscaling

Nous allons enfin tester le script d'autoscaling développé pour ajuster dynamiquement le nombre de réplicas en fonction de l'utilisation CPU de la machine hôte.

Pour rappel, le script est configuré pour augmenter le nombre de réplicas de 1 si l'utilisation CPU dépasse 80%, et pour réduire le nombre de réplicas de 1 si l'utilisation CPU descend en dessous de 30%.

Comme nous pouvons l'observer dans la figure A.5, le script réagit correctement aux variations de charge : lorsque l'utilisation CPU dépasse 80%, le script augmente le nombre de réplicas, ce qui a pour effet de produire un pic de latence dû à la création du nouveau conteneur à partir de 40 secondes, mais qui permet ensuite, une fois le replica déployé, de réduire la latence à des valeurs similaires à celles observées avec 2 replicas.

3.2 Discussion

3.2.1 Adéquation aux besoins

Les résultats obtenus confirment que Dokploy répond aux objectifs initialement fixés, le déploiement depuis un dépôt Git s'effectue sans intervention manuelle sur la machine hôte, la gestion multi-utilisateurs permet une séparation effective des projets, et le dashboard offre une visibilité suffisante sur les logs et l'état des applications.

Du point de vue pédagogique, la plateforme remplit donc son rôle : un étudiant peut déployer une application sans manipuler directement l'infrastructure sous-jacente.

3.2.2 Limites matérielles

La contrainte la plus structurante reste celle des ressources, avec seulement 2 vCPU et 4,2 Go de RAM, la plateforme ne dispose d'aucune marge d'élasticité réelle.

Les résultats de stress test l'illustrent clairement : une seule instance du serveur RNN consomme environ 50 % du CPU disponible, ce qui laisse peu de place pour des déploiements simultanés.

Le passage à trois réplicas s'est révélé contre-productif, saturant la machine au point de provoquer un recours probable au swap et de faire bondir la latence.

Ce constat n'est pas un échec de Dokploy, mais une illustration concrète d'une limite fondamentale du self-hosting : contrairement à un cloud public, on ne peut pas abstraire l'absence de ressources physiques.

Enfin on a pu remarquer que bien qu'ayant pour effet de réduire la latence, le scaling horizontal ne fait que redistribuer une ressource CPU fixe, sans l'augmenter.

3.2.3 Limites de Dokploy

Dokploy présente également des limites intrinsèques, l'absence d'autoscaling natif oblige à recourir à des scripts externes comme celui développé dans ce projet.

Cette approche fonctionne, mais reste fragile : elle repose sur une métrique unique (le CPU global de la machine hôte) plutôt que sur des indicateurs applicatifs précis comme

la latence ou le nombre de requêtes en file d'attente, ce qui limite sa réactivité et sa précision. Par ailleurs, la synchronisation des réplicas autour du modèle actif a nécessité une modification architecturale du serveur RNN lui-même, chaque replica consultant la base de données avant chaque prédiction pour détecter un changement de modèle.

Cette solution fonctionne, mais introduit une latence supplémentaire et une dépendance à la base de données sur le chemin critique de chaque requête.

3.3 Perspectives

Plusieurs axes d'amélioration peuvent être envisagés.

À court terme, une augmentation des ressources du VPS (passage à 4 vCPU et 8 Go de RAM) permettrait de lever les principales contraintes matérielles observées et d'accueillir davantage d'applications simultanées.

À moyen terme, l'ajout d'une machine secondaire permettrait de distribuer les charges entre deux nœuds, ce que Dokploy supporte nativement. Cela apporterait une forme de résilience sans nécessiter de migration vers un outil plus complexe.

Enfin, **si les besoins venaient à croître significativement** (notamment en termes de nombre d'utilisateurs ou de criticité des applications) une migration vers une solution d'orchestration plus robuste comme k3s, couplée à une interface de déploiement dédiée, constituerait une évolution naturelle.

4. Conclusion

Ce projet avait pour objectif de concevoir et déployer une plateforme de type PaaS auto-hébergée, en remplacement du service Render utilisé dans le cours d'IoT du projet DS4H. L'enjeu n'était pas simplement de substituer un outil par un autre, mais de reproduire, dans un contexte contraint, les mécanismes fondamentaux d'un environnement cloud : abstraction de l'infrastructure, déploiement automatisé, isolation des applications, gestion multi-utilisateurs.

La démarche adoptée a consisté à partir des besoins concrets pour évaluer méthodiquement plusieurs solutions existantes, avant de retenir Dokploy comme outil le mieux adapté au contexte. Les expérimentations menées ont confirmé la viabilité de cette approche : le déploiement depuis un dépôt Git s'effectue sans intervention manuelle, le scaling horizontal est fonctionnel.

D'un point de vue technique, plusieurs axes restent ouverts. L'autoscaling par substitution de modèle, esquissé en fin de projet, constituerait une réponse originale à la saturation des ressources : plutôt que d'ajouter des instances, adapter dynamiquement la complexité du modèle en fonction de la charge. Cette approche, plus proche de ce que font les systèmes embarqués ou les pipelines ML en production, mériterait d'être plus explorée.

A. Annexes

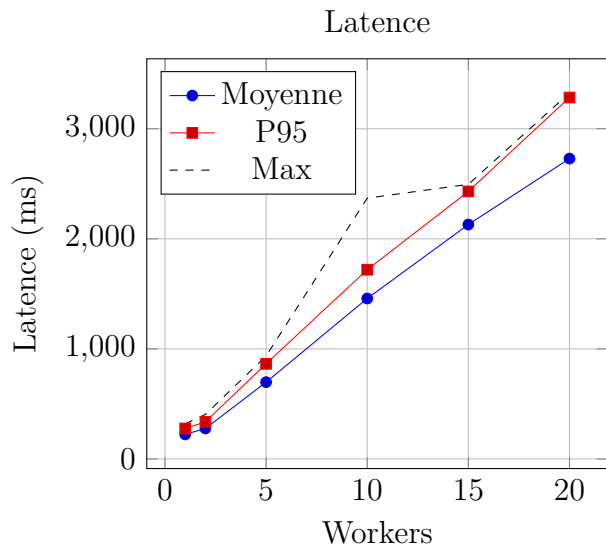


FIGURE A.1 – Latence

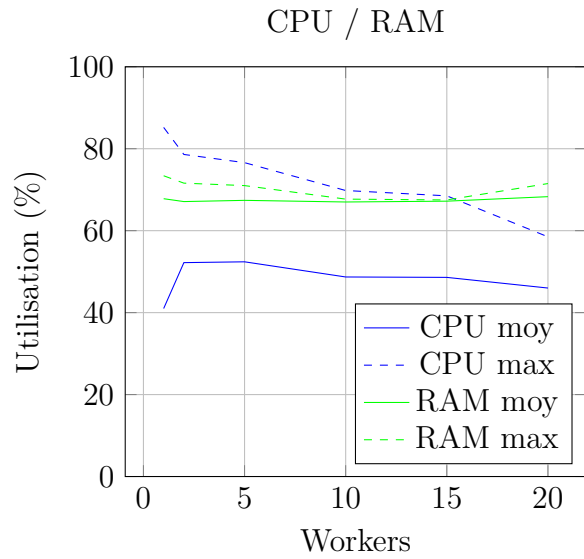


FIGURE A.2 – CPU / RAM

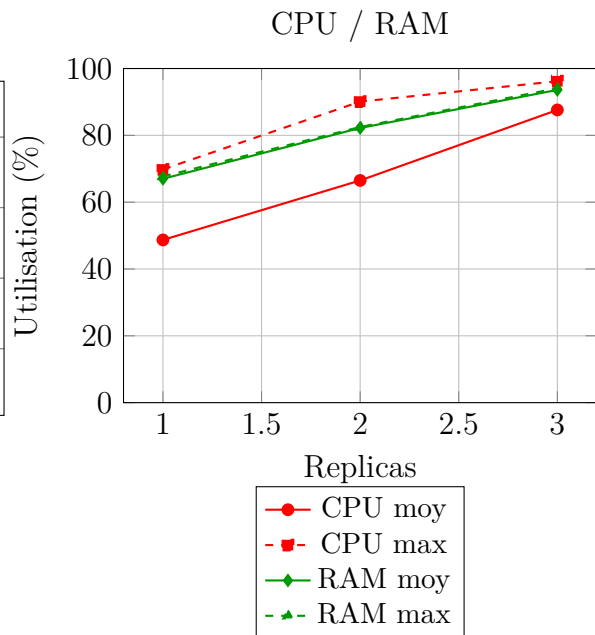
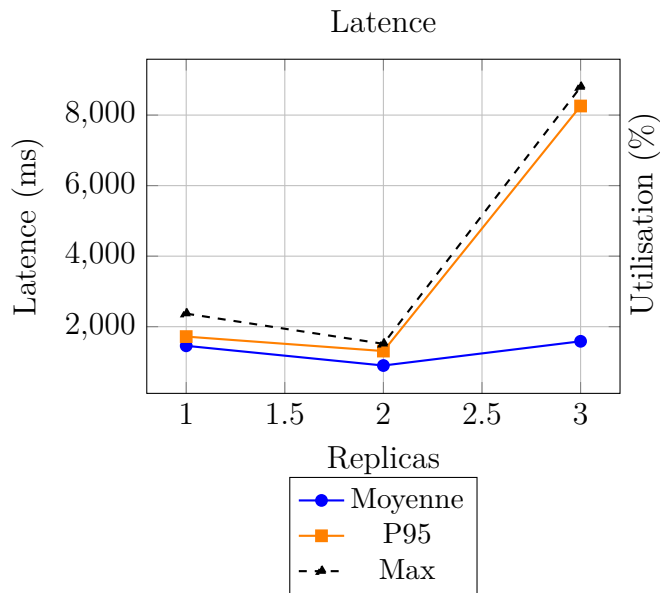


FIGURE A.3 – Impact du nombre de replicas sur la latence et les ressources

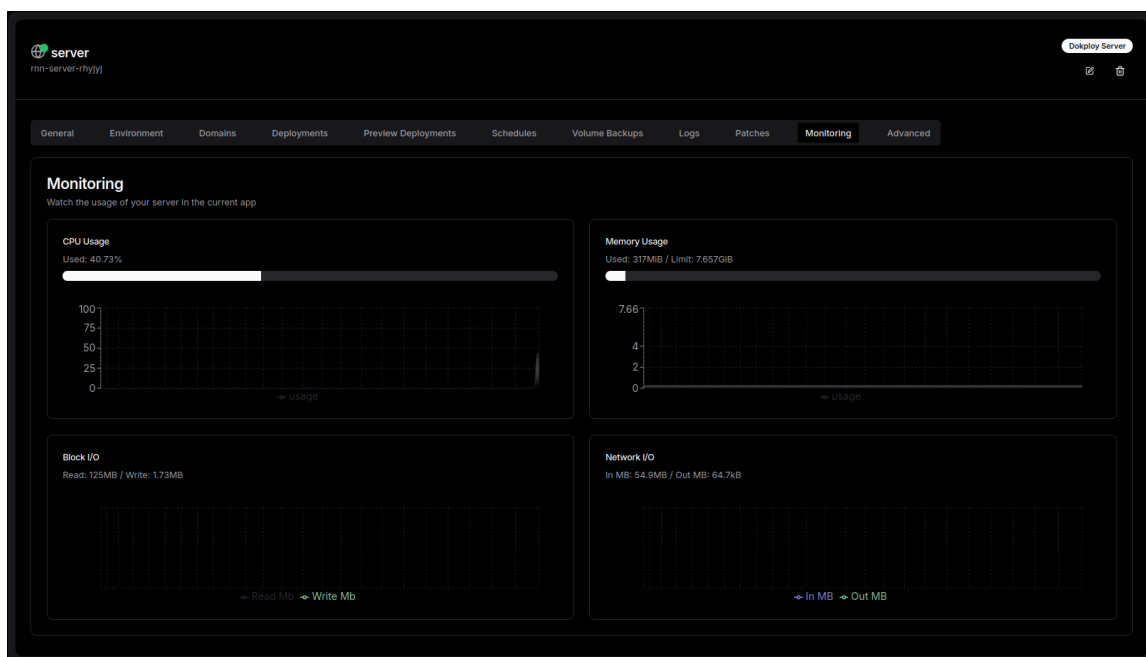


FIGURE A.4 – Dashboard de monitoring de Dokploy

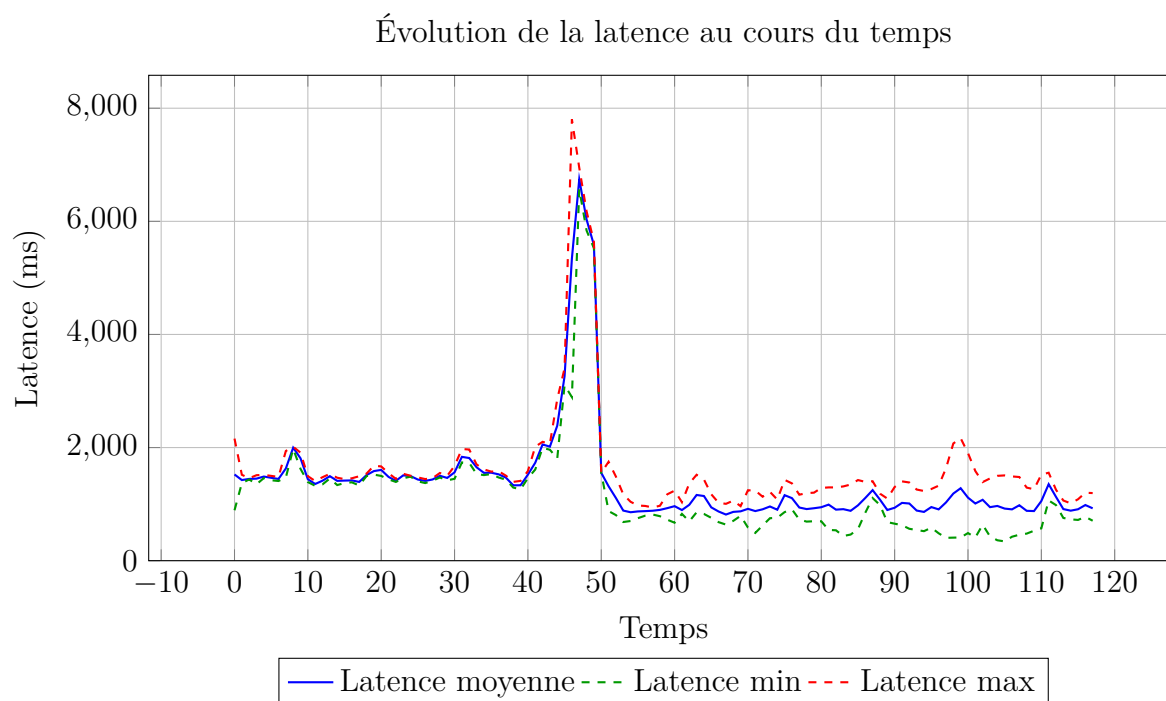


FIGURE A.5 – Évolution des latences min, moyenne et max

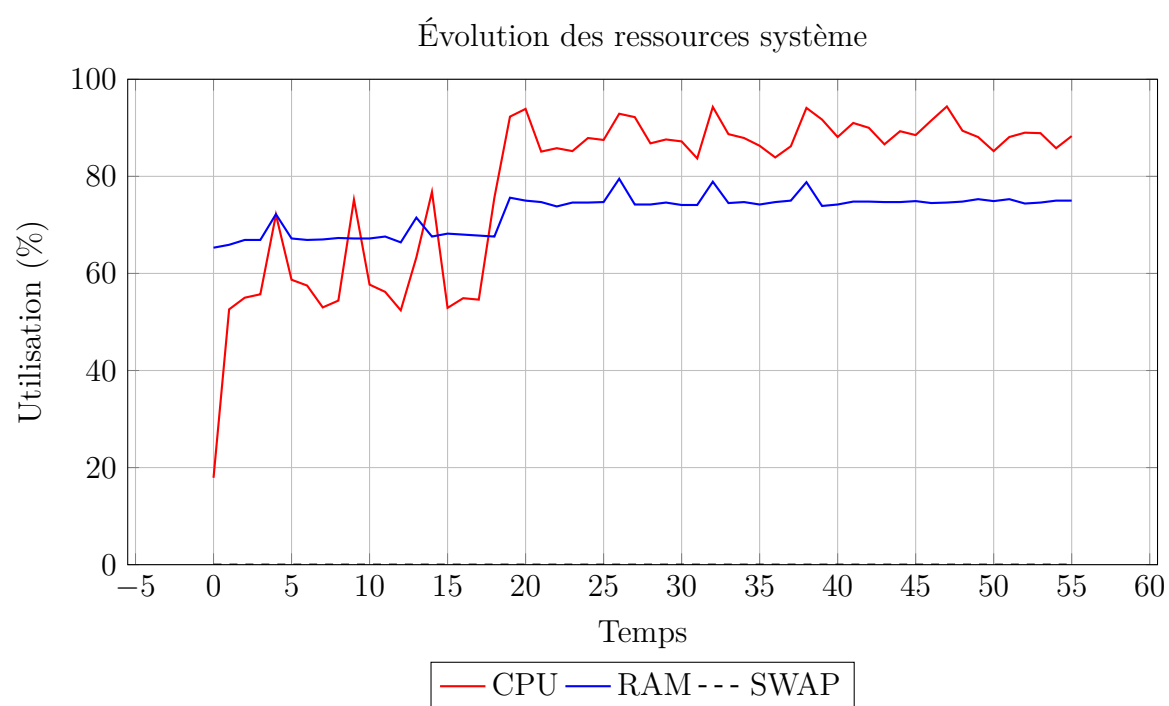


FIGURE A.6 – Évolution de l'utilisation CPU, RAM et SWAP

Bibliographie

- [1] CapRover – scalable, free and self-hosted PaaS. <https://caprover.com>. Consulté le 12 mai 2026.
- [2] Coolify – self-hosting platform with superpowers. <https://coolify.io>. Consulté le 12 mai 2026.
- [3] Docker Docs – deploy services to a swarm. <https://docs.docker.com/engine/swarm/services/>. Consulté le 12 mai 2026.
- [4] Docker Swarm – scheduler strategies. <https://github.com/atlassian/swarm/blob/master/docs/scheduler/strategy.md>. Consulté le 12 mai 2026.
- [5] Docker Swarm – swarm mode overview. <https://docs.docker.com/engine/swarm/>. Consulté le 12 mai 2026.
- [6] Dokku – the smallest PaaS implementation you’ve ever seen. <https://dokku.com/docs/getting-started/installation/>. Consulté le 12 mai 2026.
- [7] K3s – lightweight Kubernetes. <https://k3s.io>. Consulté le 12 mai 2026.
- [8] OpenStack – open source cloud computing platform. <https://www.openstack.org/software/>. Consulté le 12 mai 2026.
- [9] Len Bass, Ingo Weber, and Liming Zhu. *DevOps : A Software Architect’s Perspective*. Addison-Wesley, 2015.
- [10] Computer History Museum. George stibitz and the first remote computer, 2012.
- [11] Fernando J. Corbató and Victor A. Vyssotsky. The multics system : An examination of its structure, 1965. MIT Project MAC.
- [12] Docker Documentation. Docker swarm overview. 2026.
- [13] Ian Foster, Carl Kesselman, and Steven Tuecke. The anatomy of the grid : Enabling scalable virtual organizations. *International Journal of High Performance Computing Applications*, 15(3) :200–222, 2001.
- [14] ISO/IEC. Information technology – cloud computing – overview and vocabulary, 2014. ISO/IEC 17788 :2014.
- [15] Kubernetes Documentation. Horizontal pod autoscaling walkthrough. 2025.
- [16] Peter Mell and Timothy Grance. The nist definition of cloud computing, 2011. NIST Special Publication 800-145.
- [17] Dirk Merkel. Docker : Lightweight linux containers for consistent development and deployment. *Linux Journal*, (239), 2014.
- [18] Kief Morris. *Infrastructure as Code : Managing Servers in the Cloud*. O’Reilly Media, 2016.
- [19] Salesforce Inc. Salesforce company history, 1999.
- [20] VMware Inc. Understanding full virtualization, paravirtualization, and hardware assist, 2007.